Final Presentation

Topic: Distributed Mixture-of-Experts (MoE) systems

Phuong Ngo Pitkänen and Amirreza Jafariandehkordi

Introduction: MoE Systems

What is a Mixture-of-Experts (MoE)?

- Neural network architecture that activates only a subset of parameters per input
- Consists of multiple specialized sub-networks called *experts*
- A routing network selects which experts process each token

Why MoE matters

- Enables scaling to very large models
- Widely used in modern LLMs
- Provides improved efficiency compared to dense models

Basic architecture

- Input tokens → Router → Selected experts → Aggregated output
- Experts are distributed across GPUs in large-scale systems

Core Problem in MoE Training

Key challenge: Expert Load Imbalance

- Router may assign uneven numbers of tokens to experts
- Some experts become overloaded while others are idle

Impact on training

- Overloaded GPUs become bottlenecks
- Underloaded GPUs remain idle: wasted resources

Leads to:

- Lower training throughput
- Poor GPU utilization
- Increased communication among GPUs

Motivation

- Improve efficiency of MoE training
- Reduce imbalance and idle time
- Optimize expert placement across GPUs

Research Objectives

Learning goals

- Understand MoE architectures and routing mechanisms
- Study distributed MoE frameworks (e.g., DeepSpeed)
- Analyze performance in LUMI

Primary focus

- Expert load balancing in distributed MoE systems
- Measure imbalance using metrics
- Improve efficiency through better expert distribution

Our approach

- Benchmark baseline MoE training to identify bottlenecks
- Design a routing-aware expert rebalancing method
- Evaluate improvements using metrics

Expected outcomes

- Better GPU utilization and reduced idle time
- Improved training and scalability
- A better and more efficient MoE model

Phase 1: Benchmarking Setup

Goal of Phase 1

- Build a reproducible DeepSpeed baseline before introducing load balancing
- Measure system behavior and identify bottlenecks in training

Hardware and framework

- Experiments run on CSC LUMI
- GPUs: AMD Instinct MI250X
- Framework: PyTorch + DeepSpeed MoE

Baseline configuration

- 8 experts, top-1 routing
- 2 layers, hidden size 512
- Sequence length 512, micro-batch size 8
- 200 training steps

Metrics collected

- Throughput (tokens/sec)
- GPU memory usage
- Expert imbalance (CV, max/min ratio)

Phase 1: Baseline Results

Scaling results	**1 GPU:** 95k tokens/sec
	2 GPUs: 195k tokens/sec
	4 GPUs: 399k tokens/sec
Key observations	Throughput increased almost linearly with more GPUs
	Step time stayed close to 0.04 s across 1, 2, and 4 GPUs
	Memory per GPU decreased as experts were distributed across more devices
Expert balance	Imbalance also improved as GPU count increased
	Expert CV decreased from 0.036 (1 GPU) to 0.022 (4 GPUs)
	This gave us a stable benchmark for later comparisons
Takeaway	Phase 1 confirmed that the training pipeline, logging, and DeepSpeed setup were working correctly

Phase 1: Why We Needed Phase 2

Limit of Phase 1

- Baseline used a relatively small setup
- They did not stress expert placement strongly enough

Heavy configuration for deeper study

- 32 experts, top-2 routing
- 8 layers, hidden size 1024
- Sequence length 1024, micro-batch size 2
- 1500 training steps

Heavy baseline results

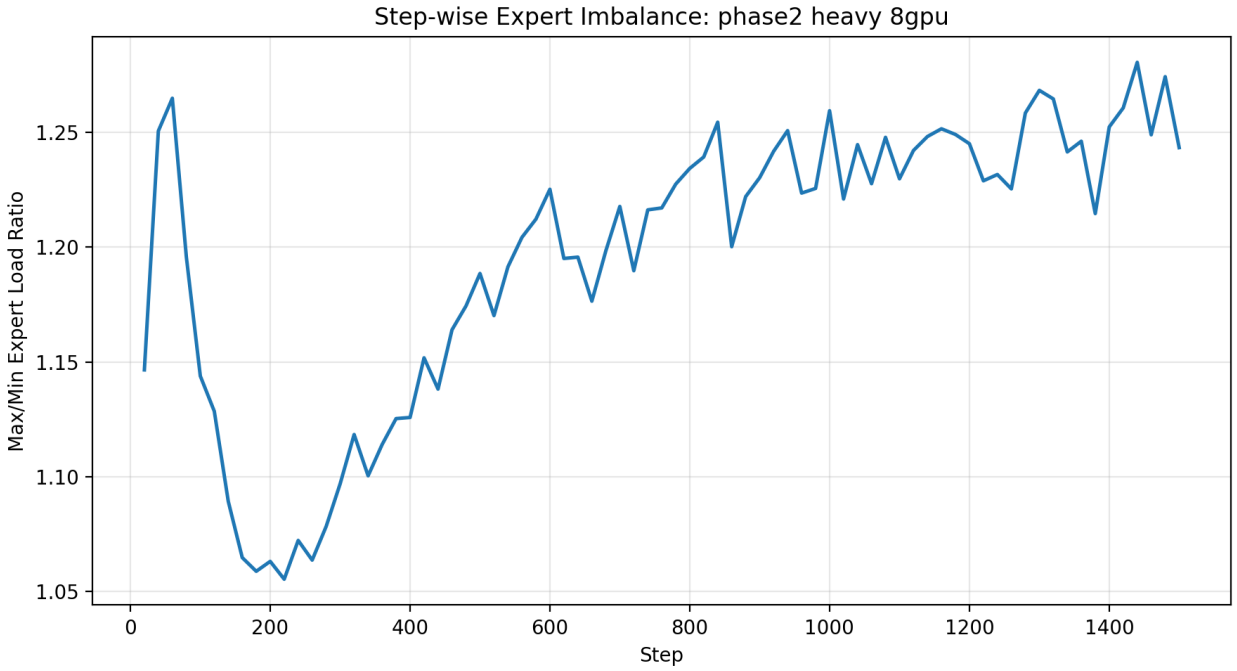
- **8 GPUs:** 46k tokens/sec
- **16 GPUs 2 Nodes:** 72.5k tokens/sec
- Expert imbalance remained noticeable:
  - CV = **0.041** on 8 GPUs
  - CV = **0.043** on 16 GPUs

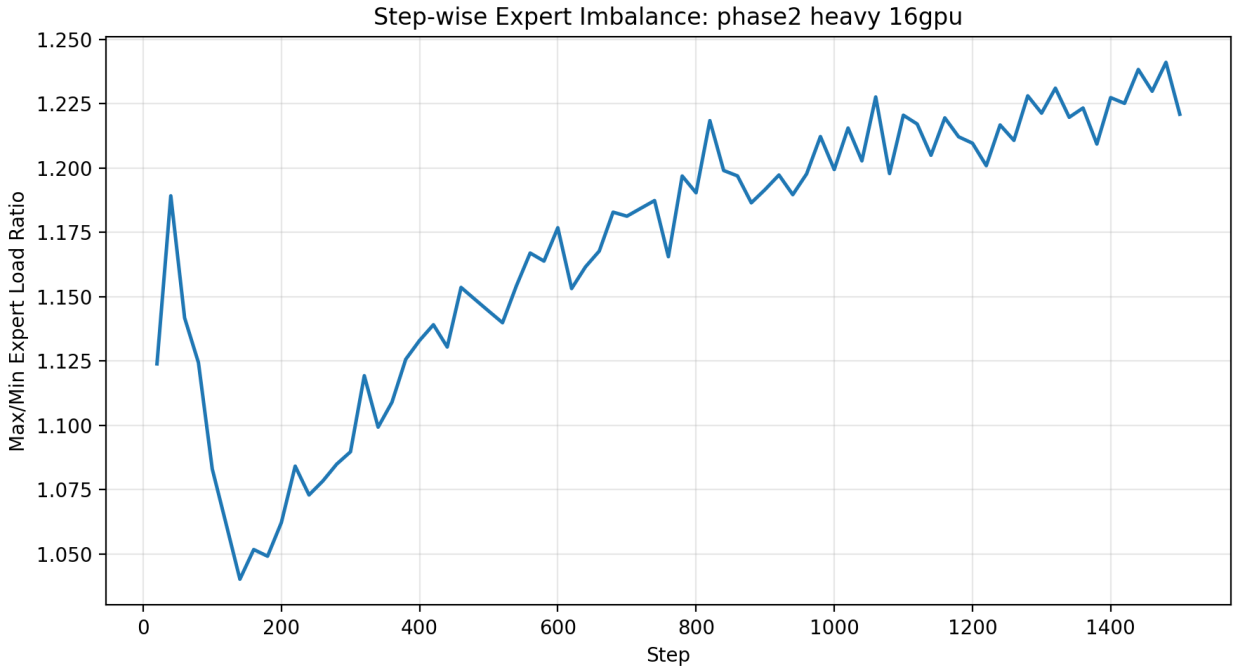
Link to Phase 2

- Reduce imbalance using routing-aware rebalancing

Phase 1: Results

Phase 1: Results





Phase 2: Load-Balancing Method

Main idea

- Use statistics during training to estimate how heavily each expert is used
- Reassign experts across GPUs to reduce imbalance

Method used over DeepSpeed

- Compute token loads per expert
- Estimate load per GPU under the current mapping
- Trigger rebalancing when the wasted fraction exceeds a threshold
- Generate a better expert-to-GPU mapping with lower imbalance and lower communication proxy

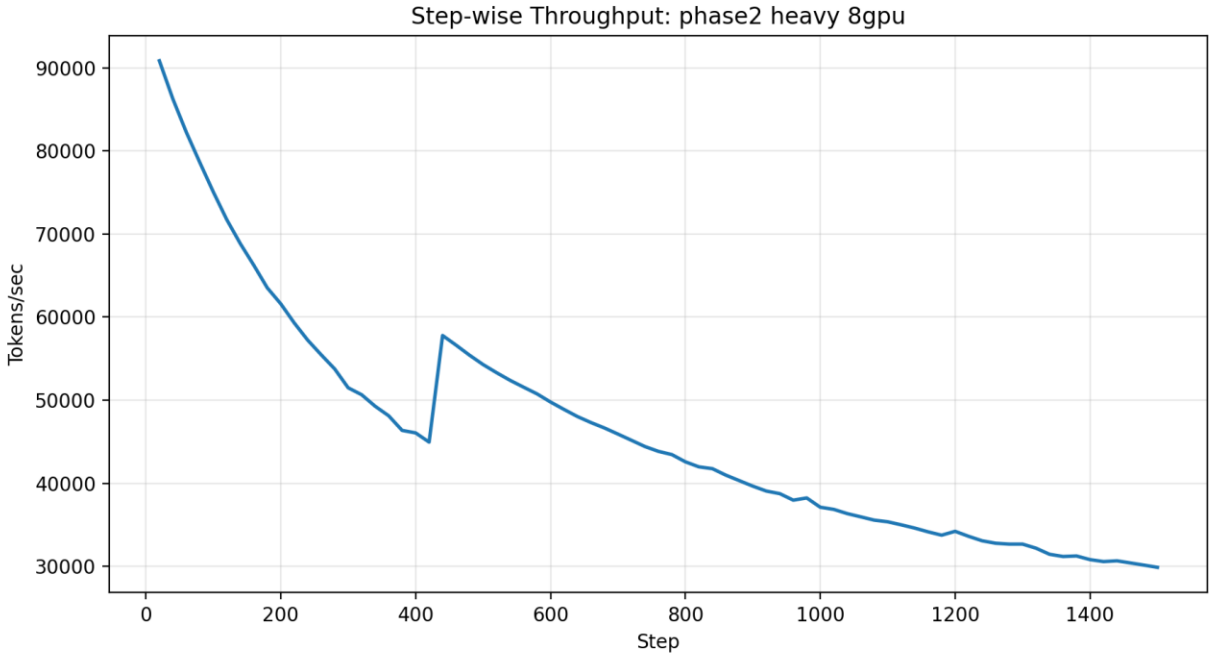
Important implementation detail

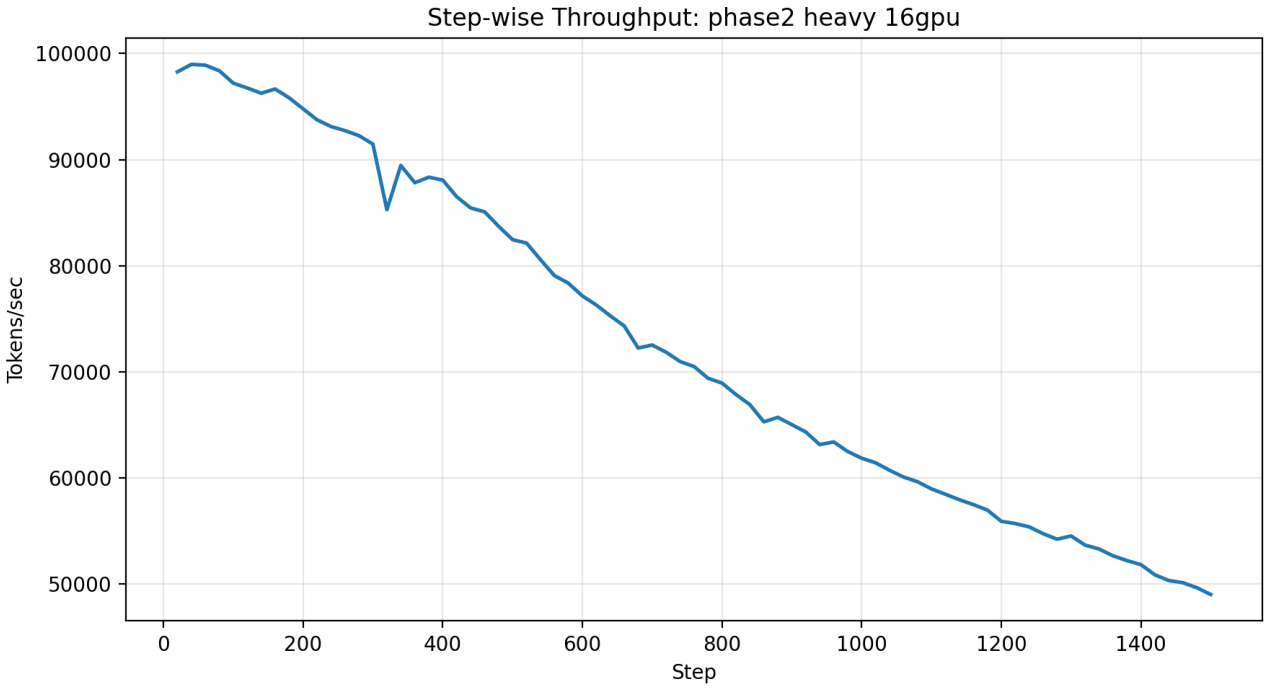
- In our DeepSpeed version, remapping is restart-based

What Phase 2 tries to improve

- Wasted fraction
- Expected imbalance after remapping
- Proxy communication cost
- Overall expert distribution across GPUs

Phase 1: Results





Phase 2: Results

Setup (8 GPUs, heavy config)

Step 3: normal run (generate proposed map)

Step 4: restart with proposed map

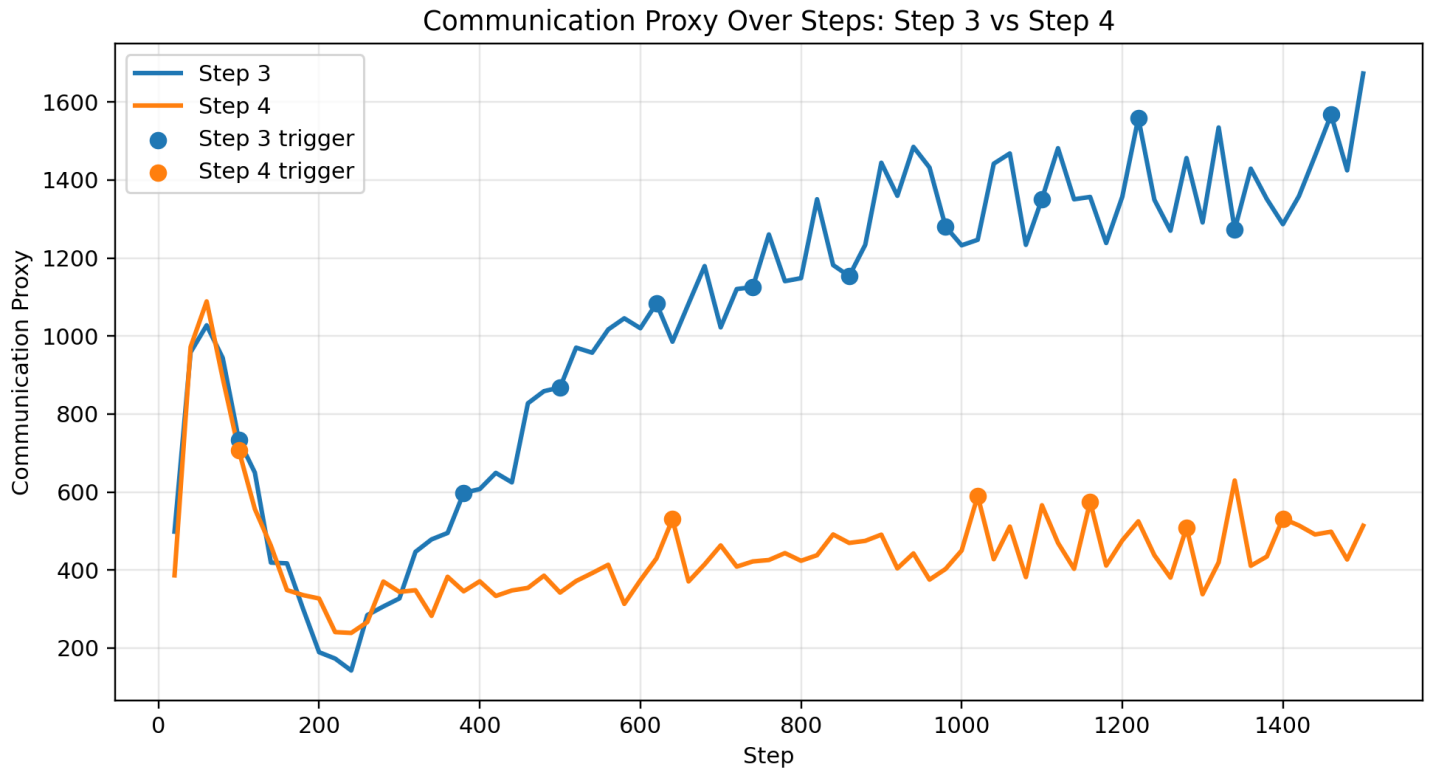
Results

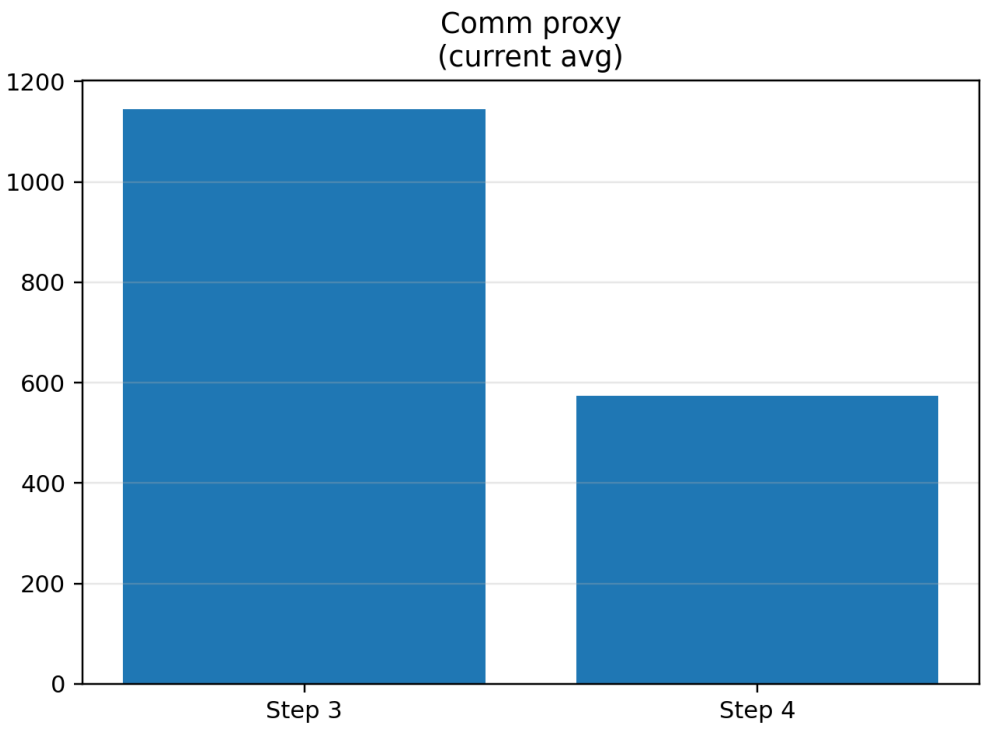
Throughput: 50.5k → 50k tokens/sec (no change)
Imbalance (CV): 0.0475 → 0.0445
Communication proxy: 1145 → 574
Rebalance triggers: 11 → 6
Expected improvement: 0.0285 → 0.0109

Takeaway

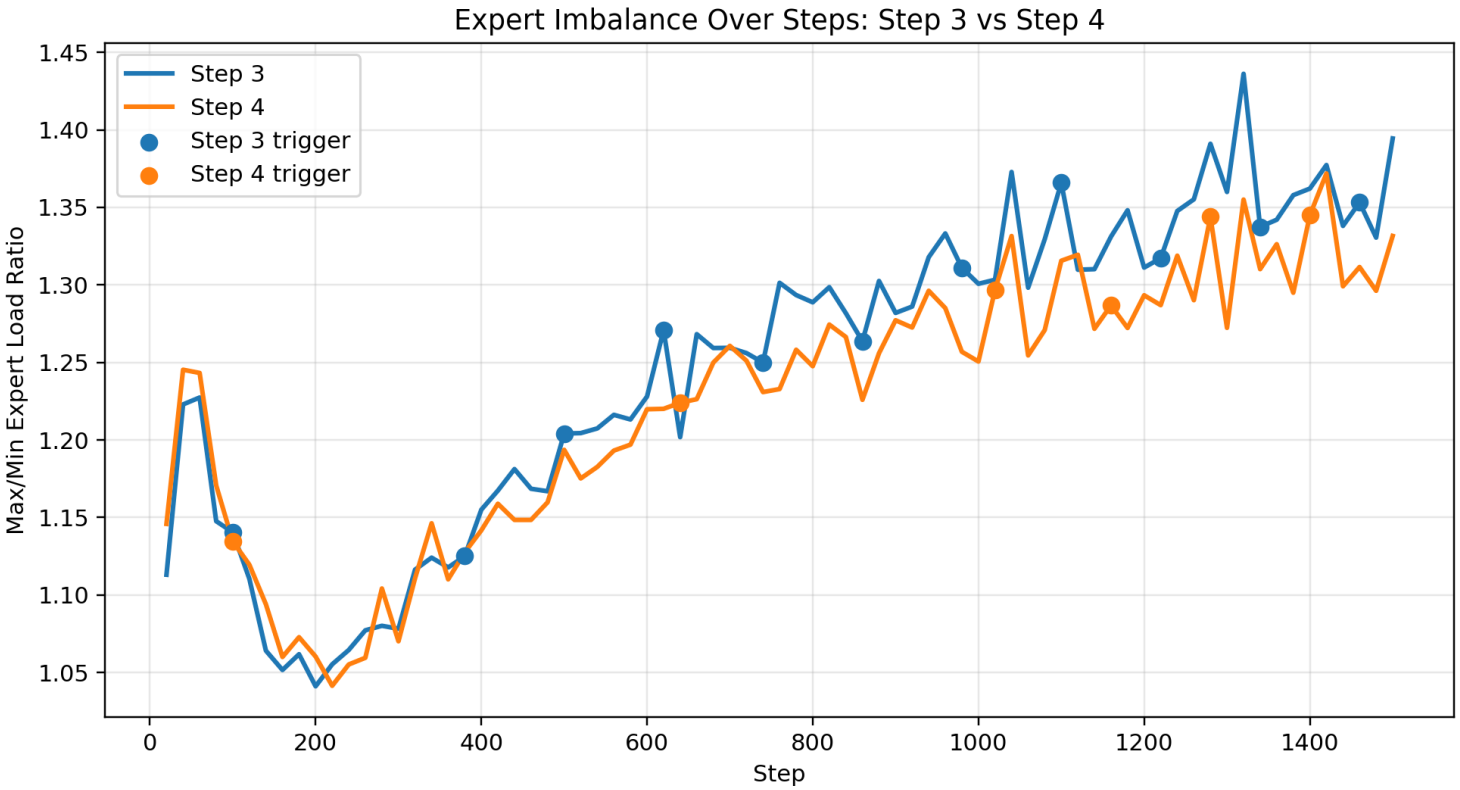
Better expert placement
Lower imbalance and communication cost
Same throughput, but more efficient system state

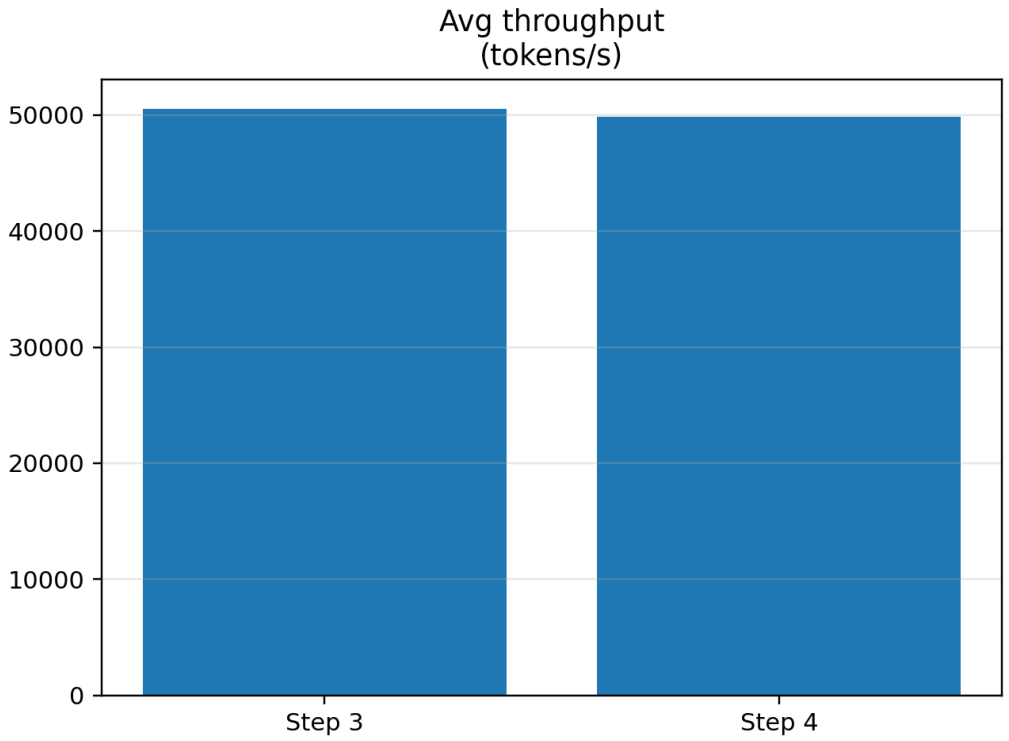
Phase 2: Results



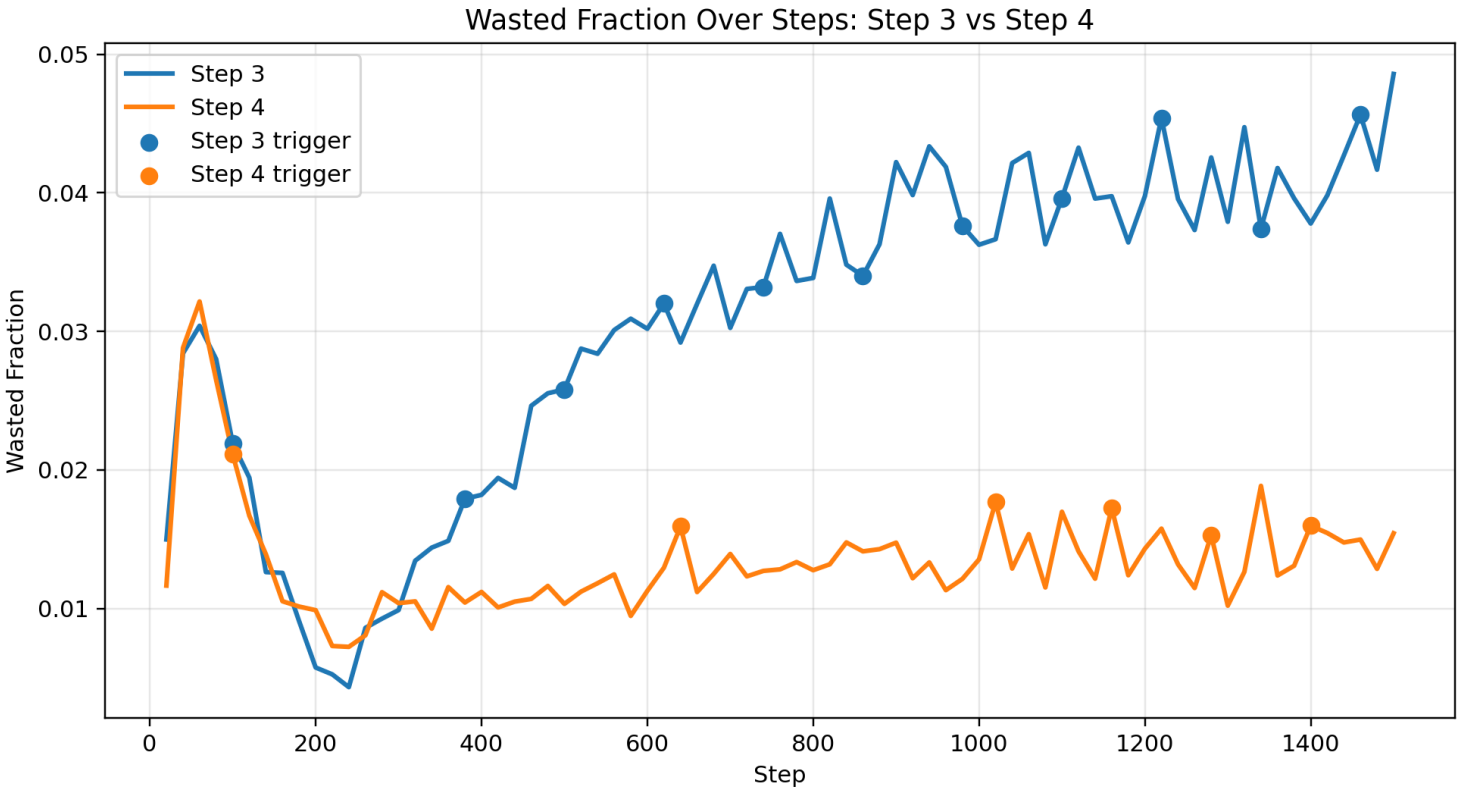


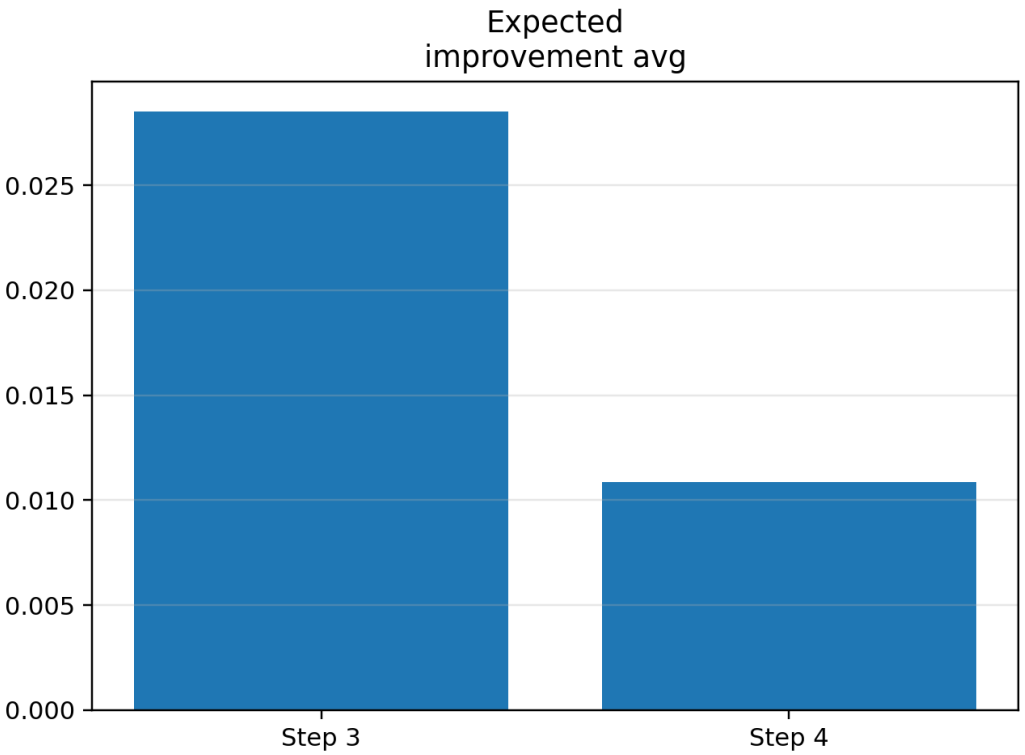
Phase 2: Results





Phase 2: Results





Parameter Study: Effects of configurations on systems performance

Sweeping through **5,400 configurations across 5 runs**: 2 GPU, 4 GPU, 8 GPU (heavy), 8 GPU (DeepSpeed map), 16 GPU (heavy)

Methodology

1. **Offline replay**. Training logs from five experimental runs are replayed step-by-step through the simulated planner.
2. **Trigger decision.** At each eval step, the planner checks whether the chosen metric exceeds θ and whether the cooldown has elapsed.
3. **Savings estimation**. Upon trigger, a proposed mapping is generated via greedy load-balancing; savings are estimated using the calibrated cost model.
4. **Aggregation.** Results are aggregated per configuration across all steps and analysed via Pareto frontier analysis.

Experimental setup

Parameters	Values tested	What it controls
Trigger metric	WF, CV, Max/mean, EMA-WF (β=0.9), Jain	*Which imbalance signal to monitor*
Threshold θ	0.03, 0.05, 0.08, 0.10	*How severe imbalance must be before acting*
Evaluation interval (Eval interval)	10, 20, 50 steps	*How often to check for imbalance*
Cooldown	50, 200, 200 steps	*Minimum rest between two triggers*
λ (comm. weight)	0.0, 0.5, 1.0	*Weight of communication cost vs compute waste*

What the parameter study set out to answer

5,400 planner configurations evaluated via offline simulation on LUMI training logs (2–16 GPUs)

1

Which signals to use?

Which imbalance signals respond to threshold tuning and which do not?

2

How much can we save?

How world size and workload severity determine recoverable latency per trigger?

3

When to check and wait?

The effect of evaluation interval and cooldown on per-trigger savings quality

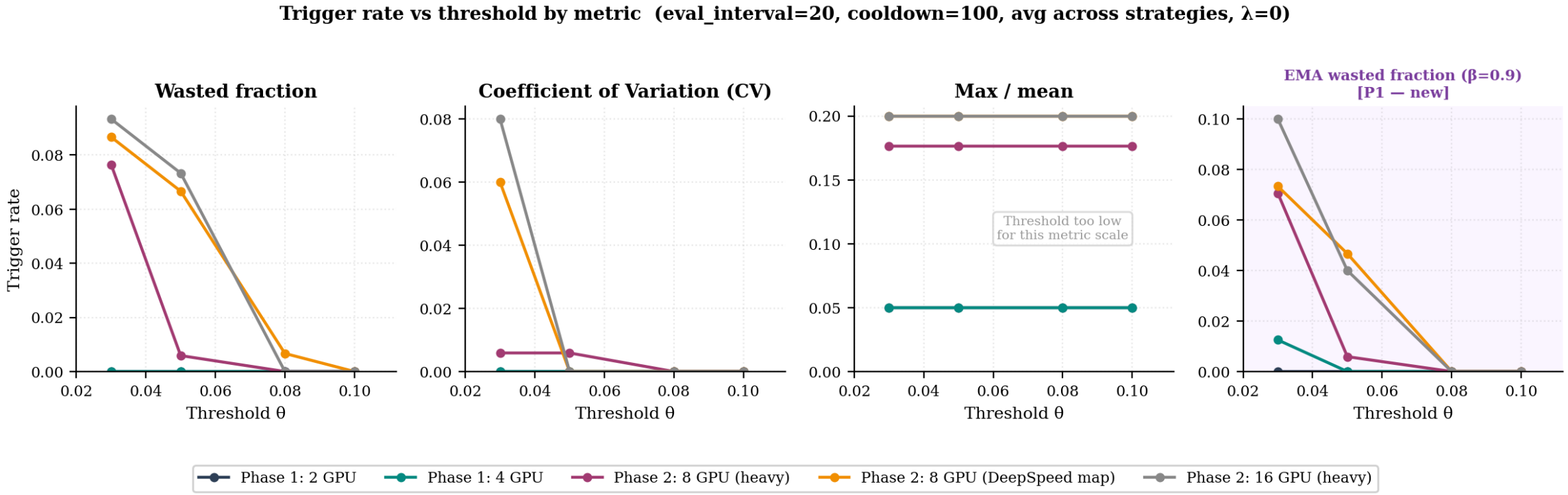
4

What is the best trade-off?

The Pareto frontier across all 5,400 configurations: trigger frequency vs savings per trigger

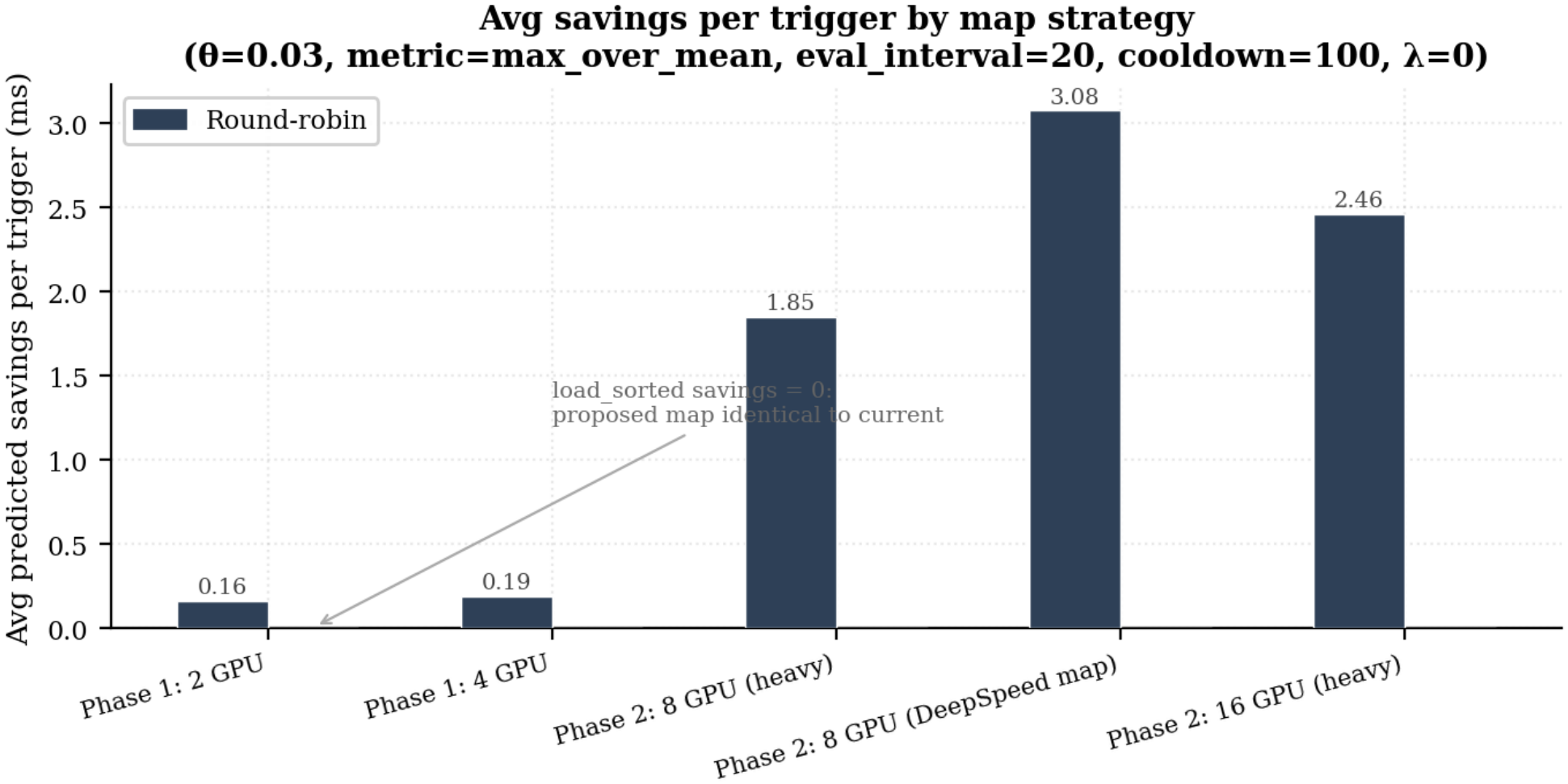
Finding 1: Not all metrics can be tuned

Question: as we raise the detection threshold, does the metric fire less often? The only metrics worth using are wasted fraction, CV, and EMA.



Finding 2: Initial expert-to-GPU mapping quality determines the upper bound of per-trigger savings throughout training.

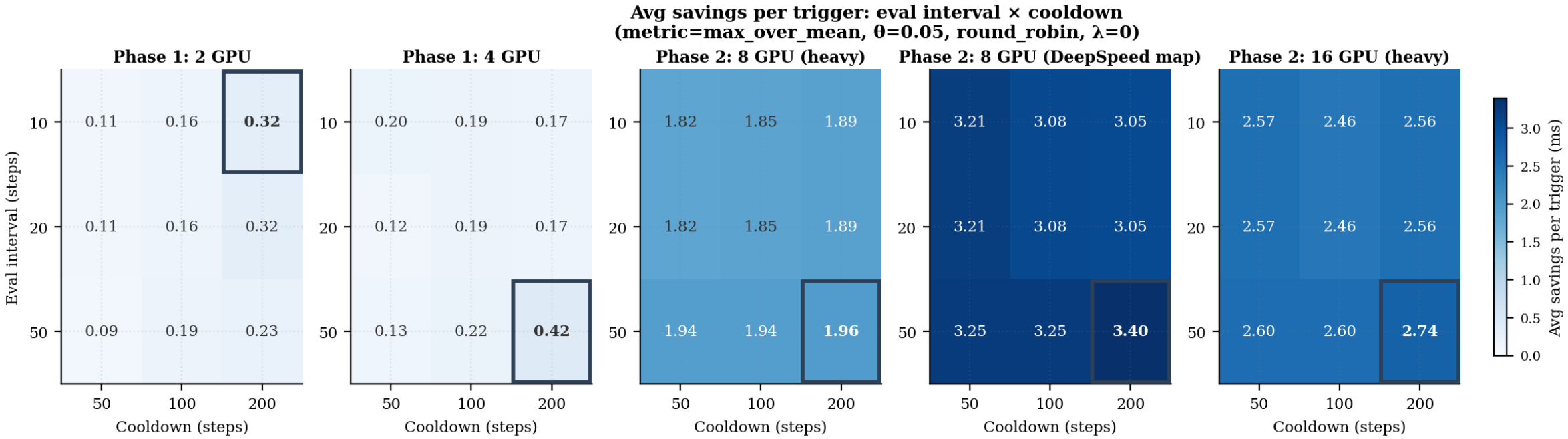
Question: if you start with round-robin and switch to load-sorted, how many milliseconds of compute waste do you save on average per rebalancing trigger.



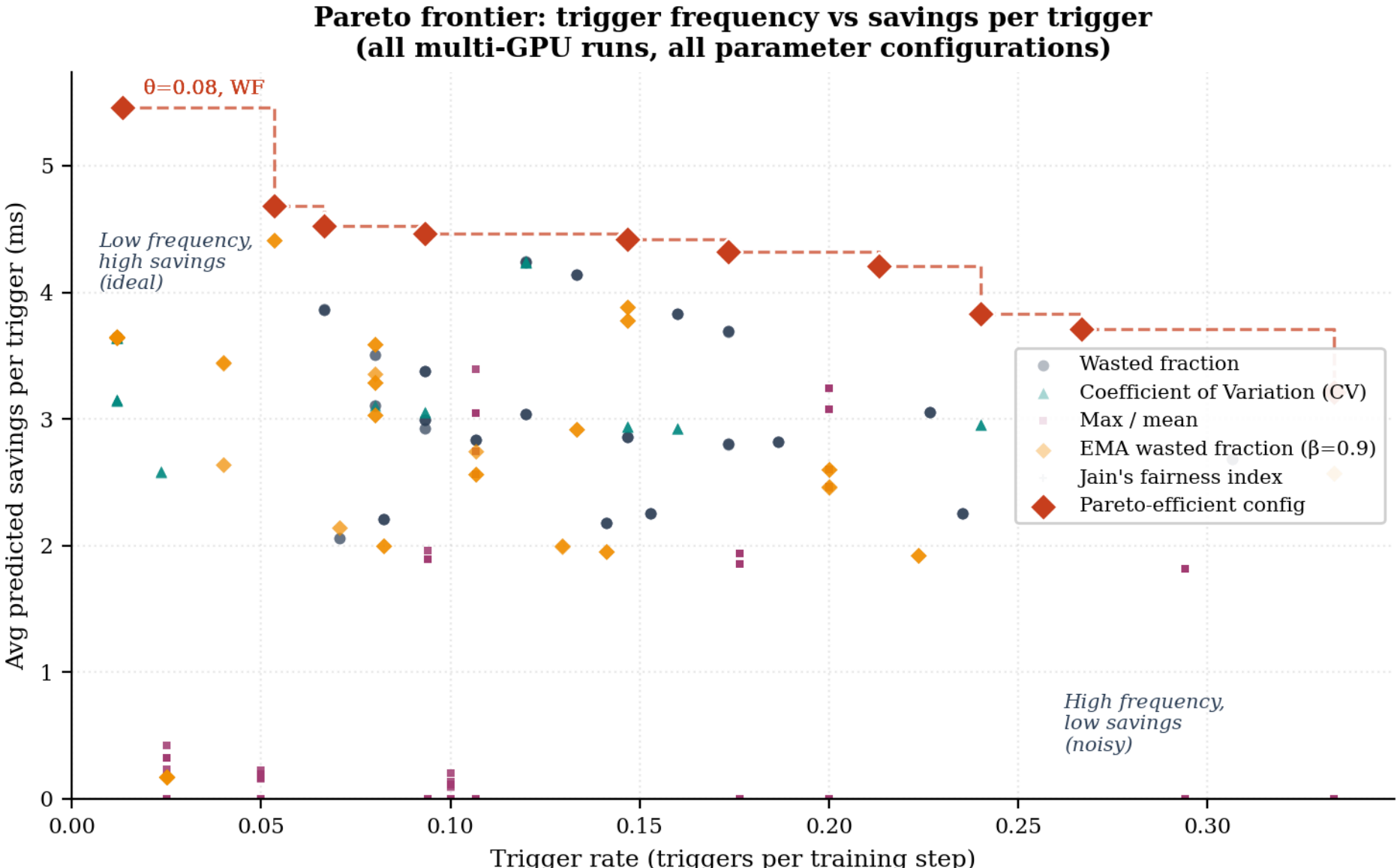
A!

Finding 3: Longer cooldown selectively suppresses low-value triggers while preserving high-value ones

Moving from cooldown 50 to cooldown 200 consistently increases average savings per trigger across all runs. When the cooldown is long, only genuinely sustained and severe imbalance events survive the wait.



Finding 4: The best overall configuration is θ=0.08 with EMA wasted fraction



Bottom line

- The only metrics worth using are wasted fraction, CV, and EMA. Max/mean cannot be tuned and Jain's index is too insensitive for this workload
- The startup assignment is not a one-time decision. How experts are placed before training begins sets the ceiling for how much every future rebalancing trigger can save
- Cooldown actively improves the quality of each trigger by filtering out marginal imbalances that would have resolved themselves
- Smoothing the signal beats lowering the threshold. EMA at $\theta=0.08$ outperforms raw wasted fraction at $\theta=0.03$ because it waits for sustained problems rather than reacting to noise at a lower bar.
- The best planner is not the most active one. the Pareto frontier shows that the configurations saving the most per trigger are precisely the ones that fire the least often

Achievements and Lessons

Achievements

- Set up the baseline model before load balancing for benchmarking
- Implemented cost model and integrated it into the rebalancing algorithm
- Ran models across different GPUs, significantly 16-GPUs
- Implemented load balancing method on 8-GPU runs by experts-to-GPUs remapping over DeepSpeed
- Gained insights into how the MoE systems are performing and what kind of impacts different parameters and metrics can have on the MoE systems' performance
- Formulated best overall configurations of parameter in which the MoE systems perform efficiently and impactfully

Challenges

- Sometimes unclear about the responsibilities and the directions we must head to for the research tasks
- Faced with multi-node port collisions.
- Unfamiliarity with the LUMI HPC at first despite training and presentation before actual use.
- What can be done more is adaptive updates during training and run 16-GPU rebalancing job to validate the sensitivity tablet at scale

A!

Kiitos
aalto.fi